

UNIVERSIDAD POLITÉCNICA SALESIANA  
MAESTRÍA EN INGENIERÍA DE SOFTWARE  
Desarrollo de Aplicaciones Empresariales

---

# Sistema de Reserva de Canchas Deportivas

Manual de Despliegue

---

## **Autores**

Pablo Jara Muñoz  
pablojaramunoz@gmail.com

Miguel Lara  
miguel.larasj@gmail.com

Sebastián Uyaguari  
sebastianuyaguari@gmail.com

Adrián Espinoza  
adrian.espinoza1292@gmail.com

## **Docente**

Christian Merchán Millán

29 de agosto de 2026

## Sobre este manual

Este documento corresponde al **entregable E5** del proyecto integrador y describe cómo levantar, verificar y operar el Sistema de Reserva de Canchas Deportivas.

Se ha escrito pensando en quien recibe el proyecto sin haber participado en él: alguien que clona el repositorio, quiere verlo funcionando en su propia máquina y necesita saber qué hacer cuando algo no arranca. Por eso el texto va directo a los comandos y deja fuera las justificaciones de diseño.

Los otros dos documentos del proyecto cubren lo que aquí se omite:

- El **informe** (entregable E1) explica la arquitectura, el modelo de datos y las reglas de negocio: *qué* hace cada pieza y *por qué* está construida así.
- El **manual de usuario** explica cómo se opera la aplicación desde el navegador, tanto para el usuario final como para el administrador.

### Nota

Todo lo que sigue está verificado sobre la versión del sistema entregada. Los comandos se ejecutan desde la raíz del repositorio, salvo que se indique otra cosa.

## Índice

|                                                     |          |
|-----------------------------------------------------|----------|
| <b>Sobre este manual</b>                            | <b>i</b> |
| <b>1. Requisito único</b>                           | <b>1</b> |
| <b>2. Levantar el sistema</b>                       | <b>1</b> |
| 2.1. Si algún puerto está ocupado . . . . .         | 1        |
| <b>3. Qué se levanta</b>                            | <b>2</b> |
| <b>4. Usuarios de prueba</b>                        | <b>3</b> |
| <b>5. Verificar que levantó bien</b>                | <b>3</b> |
| <b>6. Operación</b>                                 | <b>4</b> |
| 6.1. Arranque limpio . . . . .                      | 4        |
| 6.2. Detener sin borrar . . . . .                   | 5        |
| 6.3. Ver los registros . . . . .                    | 5        |
| 6.4. Reconstruir tras un cambio de código . . . . . | 5        |
| 6.5. Trabajar solo en el frontend . . . . .         | 5        |
| <b>7. Configuración</b>                             | <b>5</b> |
| <b>8. Problemas frecuentes</b>                      | <b>6</b> |
| <b>9. Ejecutar las pruebas</b>                      | <b>7</b> |
| 9.1. La prueba de concurrencia . . . . .            | 8        |

## 1. Requisito único

**Docker 24 o superior, con Compose v2.** Nada más.

No hace falta instalar Java, Node, Maven ni PostgreSQL: todo se compila y se ejecuta dentro de contenedores. Es una decisión deliberada —el Principio VII de la constitución del proyecto exige que el sistema levante con un solo comando— y también lo que hace que la evaluación no dependa de qué tenga instalado quien evalúa.

Para comprobar la versión instalada:

```
docker --version      # Docker version 24.x o superior
docker compose version # Docker Compose version v2.x
```

## 2. Levantar el sistema

```
git clone <url-del-repositorio> && cd reserva-canchas
docker compose up
```

Y abrir `http://localhost`.

El primer arranque compila las imágenes y tarda varios minutos; los siguientes son cuestión de segundos, porque Docker reutiliza las capas ya construidas.

Para dejarlo corriendo en segundo plano y que el comando no devuelva el control hasta que todo esté sano:

```
docker compose up -d --wait
```

Cuando termina, `docker compose ps` debe mostrar los once servicios en estado `healthy`.

### 2.1. Si algún puerto está ocupado

El gateway se publica en el puerto 8080 del anfitrión. Si esa máquina ya lo usa, el arranque falla con `port is already allocated`. Se cambia sin tocar el archivo de Compose, definiendo la variable al vuelo:

```
GATEWAY_PORT=18080 docker compose up -d --wait
```

Todos los puertos publicados admiten esa sustitución:

|                        |                  |               |
|------------------------|------------------|---------------|
| SHELL_PORT             | MS_USUARIOS_PORT | GATEWAY_PORT  |
| MF_RESERVAS_PORT       | MS_CANCHAS_PORT  | POSTGRES_PORT |
| MF_ADMINISTRACION_PORT | MS_RESERVAS_PORT | EDGE_PORT     |
| MF_REPORTES_PORT       | MS_REPORTES_PORT |               |

#### Nota

Cambiar esos puertos no afecta a la aplicación. El navegador solo usa `http://localhost`, que es el puerto 80 del *edge*; los demás puertos existen para inspeccionar cada servicio por separado.

### 3. Qué se levanta

Once contenedores, definidos en `docker-compose.yml`. Nueve se construyen desde el propio repositorio —cada uno con su `Dockerfile` en `frontend/<nombre>` o `backend/<nombre>`— y dos se descargan ya hechos:

| Servicio          | Origen                | Red                   | Puerto | Para qué                                                      |
|-------------------|-----------------------|-----------------------|--------|---------------------------------------------------------------|
| edge              | nginx<br>1.27-alpine  | frontend<br>+ backend | 80     | Único origen: sirve el shell, los remotes y <code>/api</code> |
| shell             | build local           | frontend              | 3000   | Host de los microfrontends, sesión y enrutado                 |
| mf-reservas       | build local           | frontend              | 3001   | Disponibilidad, nueva reserva, mis reservas                   |
| mf-administracion | build local           | frontend              | 3002   | Canchas, reservas y usuarios                                  |
| mf-reportes       | build local           | frontend              | 3003   | Los cuatro indicadores                                        |
| gateway           | build local           | backend               | 8080   | Verifica la credencial y rutea                                |
| ms-usuarios       | build local           | backend               | 8081   | Registro, autenticación y usuarios                            |
| ms-canchas        | build local           | backend               | 8082   | Catálogo, horarios y bloques                                  |
| ms-reservas       | build local           | backend               | 8083   | Reservas y disponibilidad                                     |
| ms-reportes       | build local           | backend               | 8084   | Indicadores; <b>sin base de datos</b>                         |
| postgres          | postgres<br>16-alpine | backend               | 5432   | Las tres bases, aisladas por credenciales                     |

El **edge** es el **único contenedor conectado a las dos redes**. Es lo que impide que el navegador alcance un microservicio saltándose el gateway: los servicios de la red *backend* sencillamente no son visibles desde fuera.

Los puertos de los microservicios se publican solo para poder inspeccionar su Swagger durante la evaluación; la aplicación no los usa.

## 4. Usuarios de prueba

Los siembra la migración `V2__seed_usuarios.sql` en el primer arranque.

| Usuario  | Contraseña | Rol                     | Para qué sirve                                            |
|----------|------------|-------------------------|-----------------------------------------------------------|
| admin    | admin      | ADMINISTRADOR           | Catálogo, listado global de reservas, usuarios y reportes |
| cliente1 | cliente1   | USUARIO_FINAL           | El recorrido de reserva y cancelación                     |
| cliente2 | cliente2   | USUARIO_FINAL           | El segundo lado del escenario de concurrencia             |
| inactivo | inactivo   | USUARIO_FINAL, inactivo | Demostrar el rechazo de una cuenta inactiva               |

Las contraseñas se guardan hasheadas con BCrypt; en la base no hay ninguna en claro.

### Atención

Son credenciales de demostración, y la contraseña coincide con el nombre de usuario. En un despliegue real se cambian antes de exponer el sistema.

El catálogo de canchas y unas reservas de ejemplo también se siembran. Las reservas usan **fechas relativas al arranque**, no absolutas, para que las confirmadas sigan siendo futuras y se puedan cancelar en vivo por mucho tiempo que pase desde que se escribió el seed.

## 5. Verificar que levantó bien

Tres comprobaciones bastan:

```
docker compose ps                                # los once en healthy
curl -u cliente1:cliente1 http://localhost/api/canchas # el catálogo
open http://localhost                             # la aplicación
```

La segunda es la más informativa de las tres: si devuelve el catálogo, entonces el edge, el gateway, `ms-usuarios` —que verificó la credencial—, `ms-canchas` y PostgreSQL están todos funcionando y comunicándose entre sí.

Una petición sin credenciales debe responder 401:

```
curl -i http://localhost/api/canchas
```

El Swagger de cada microservicio queda accesible por su puerto directo:

- `http://localhost:8081/swagger-ui.html` — usuarios y autenticación
- `http://localhost:8082/swagger-ui.html` — canchas
- `http://localhost:8083/swagger-ui.html` — reservas
- `http://localhost:8084/swagger-ui.html` — reportes

## 6. Operación

### 6.1. Arranque limpio

```
docker compose down -v && docker compose up -d --wait
```

El `-v` borra el volumen `pgdata`. Es lo que hace que vuelvan a ejecutarse los scripts de `infra/postgres/init/` y las migraciones con su seed, dejando el sistema exactamente como la primera vez.

#### Atención

Sin `-v` los datos se conservan entre arranques: es lo que se quiere entre sesiones de trabajo, y lo que **no** se quiere antes de una demostración, porque las reservas de ejemplo se sembraron con fechas relativas al primer arranque y con el tiempo quedan en el pasado.

## 6.2. Detener sin borrar

```
docker compose down      # para los contenedores, conserva los datos
```

## 6.3. Ver los registros

```
docker compose logs -f ms-reservas      # uno
docker compose logs -f                  # todos
```

## 6.4. Reconstruir tras un cambio de código

```
docker compose up -d --wait --build ms-reservas
```

Sin `-build`, Compose reutiliza la imagen anterior y el cambio no surte efecto. Es la equivocación más fácil de cometer, y la más desconcertante, porque el sistema arranca sin error alguno mostrando la versión vieja.

## 6.5. Trabajar solo en el frontend

```
docker compose up shell mf-reservas mf-administracion mf-reportes
```

Levanta los cuatro microfrontends sin el backend. Las llamadas a `/api` fallarán, que es lo esperado en ese modo.

# 7. Configuración

Todo se configura por variables de entorno, con valores por defecto que funcionan sin definir ninguna. Las que importan:

| Variable             | Por defecto  | Qué controla                                   |
|----------------------|--------------|------------------------------------------------|
| POSTGRES_PASSWORD    | postgres     | Contraseña del superusuario de PostgreSQL      |
| USUARIOS_DB_PASSWORD | usuarios_app | Contraseña del rol de <code>usuarios_db</code> |
| CANCHAS_DB_PASSWORD  | canchas_app  | Contraseña del rol de <code>canchas_db</code>  |
| RESERVAS_DB_PASSWORD | reservas_app | Contraseña del rol de <code>reservas_db</code> |

| Variable     | Por defecto | Qué controla                             |
|--------------|-------------|------------------------------------------|
| GATEWAY_PORT | 8080        | Puerto del gateway en el anfitrión       |
| EDGE_PORT    | 80          | Puerto por el que se sirve la aplicación |

Se definen en un archivo `.env` en la raíz del repositorio, que Compose lee por su cuenta:

```
POSTGRES_PASSWORD=una-contraseña-larga
USUARIOS_DB_PASSWORD=otra
CANCHAS_DB_PASSWORD=otra-más
RESERVAS_DB_PASSWORD=y-otra
```

**Ese archivo no se versiona:** está en `.gitignore`, de modo que ninguna credencial llega al repositorio.

#### Atención

Las contraseñas de base se aplican **al crear el volumen**. Si se cambian con `pgdata` ya existente, los servicios no podrán conectarse y reiniciarán en bucle. Hay que hacer `docker compose down -v` antes.

Cada microservicio recibe **solo la credencial de su propia base**. `ms-reportes` no recibe ninguna porque no tiene base de datos: obtiene todo por HTTP de `ms-reservas` y `ms-canchas`.

## 8. Problemas frecuentes

| Síntoma                                            | Causa                                       | Solución                                                                   |
|----------------------------------------------------|---------------------------------------------|----------------------------------------------------------------------------|
| <code>port is already allocated</code> al arrancar | Otro proceso usa ese puerto                 | <code>GATEWAY_PORT=18080 docker compose up -d</code>                       |
| <code>/api/...</code> devuelve <b>502</b>          | El gateway aún no está sano                 | Esperar; el edge resuelve los destinos en cada petición y se recupera solo |
| <code>/api/...</code> devuelve <b>503</b>          | El microservicio destino no responde        | <code>docker compose logs &lt;servicio&gt;</code>                          |
| Un microservicio reinicia en bucle                 | Flyway falló, o la base rechazó la conexión | Revisar sus registros; si se cambiaron contraseñas, <code>down -v</code>   |

| Síntoma                               | Causa                              | Solución                                                            |
|---------------------------------------|------------------------------------|---------------------------------------------------------------------|
| Los frontends quedan <b>unhealthy</b> | Su healthcheck no responde         | <code>docker compose logs shell</code>                              |
| Un cambio de código no se refleja     | Se levantó sin <code>-build</code> | <code>docker compose up -d -build &lt;servicio&gt;</code>           |
| La sesión se cierra sola              | La cuenta fue inactivada           | El gateway verifica la credencial en cada petición; volver a entrar |

#### Nota

Ante cualquier síntoma no listado, el primer comando es siempre `docker compose logs <servicio>`. Los cuatro microservicios registran el motivo real del fallo al arrancar, y Flyway es especialmente explícito cuando una migración no cuadra con el esquema existente.

## 9. Ejecutar las pruebas

Las pruebas de reglas de negocio no necesitan base de datos ni Spring, pero sí un JDK 21. Las de arranque (`*ApplicationTests`) levantan un PostgreSQL efímero con Testcontainers, así que necesitan además acceso al demonio de Docker.

Si la máquina no tiene JDK 21 instalado, todo se ejecuta dentro de un contenedor, sin instalar nada:

```
docker volume create rc-m2
docker run --rm \
  -v "$PWD/backend/ms-reservas:/build" -v rc-m2:/root/.m2 -w /build \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -e TESTCONTAINERS_HOST_OVERRIDE=host.docker.internal \
  eclipse-temurin:21-jdk-alpine ./mvnw -B test
```

El volumen `rc-m2` guarda las dependencias de Maven entre ejecuciones: sin él, cada corrida vuelve a descargarlas enteras.

Las **85 pruebas** de los cinco servicios pasan.

## 9.1. La prueba de concurrencia

La verificación de RN-02 —que dos reservas simultáneas sobre el mismo bloque no puedan confirmarse las dos— sí necesita un PostgreSQL real, porque lo que se está probando es la restricción de exclusión del motor. Se ejecuta con el sistema levantado:

```
docker run --rm --network reserva-canchas_backend \
  -v "$PWD/backend/ms-reservas:/build" -v rc-m2:/root/.m2 -w /build \
  -e RESERVAS_IT_DB_URL="jdbc:postgresql://postgres:5432/reservas_db" \
  eclipse-temurin:21-jdk-alpine \
  ./mvnw -B test -Dtest=ConcurrenciaReservaIT
```

Son diez repeticiones, y en todas exactamente una reserva se confirma y la otra se rechaza.

### Nota

La variable `RESERVAS_IT_DB_URL` es también el interruptor de esa prueba: sin ella definida, la clase se omite. Así la batería normal sigue corriendo en cualquier máquina sin exigir una base de datos levantada.